

飞虹 Code

终端 AI 编程智能体软件

用户使用手册

软件版本：V8.0.0

著作权人：吴赐虹

开发完成日期：2026年9月3日

首次发表日期：2026年9月3日

开发单位：晋江市飞虹智科技企业管理有限公司

研发中心：飞扬企源研发中心

目 录

开发者前言

第一章 软件概述

第二章 安装部署

第三章 快速上手

第四章 核心功能详解

第五章 v8.0 新增功能

第六章 Web 控制台

第七章 高级功能

第八章 配置参考

第九章 常见问题与故障排查

第十章 详细教程与实战案例

附录 A 命令速查表

附录 B 配置速查表

附录 C 术语表

附录 D 更新日志

开发者前言

我是吴赐虹，飞虹 Code 的独立开发者。从事软件开发和企业信息化服务多年，长期在一线为泉州制造业企业提供数字化转型解决方案。在日常工作中，我深刻体会到传统开发模式的痛点：需求变更频繁、重复编码工作量大、代码质量参差不齐、新人上手周期长、技术债越积越多。这些问题在中小团队中尤为突出，我们既没有大厂那样完善的工程体系，也难以承担昂贵的商业开发工具费用。

2026 年初，大语言模型技术迎来爆发式发展，我开始尝试将 AI 能力融入日常开发流程。最初只是使用在线 AI 工具辅助写代码，但很快发现这些工具有明显的局限性：无法访问本地代码库、不能执行命令验证结果、上下文窗口有限、数据安全无法保障、与现有开发工具链割裂。于是我萌生了一个想法：为什么不自己开发一个真正面向终端开发者的 AI 编程智能体呢？它应该能理解整个项目的代码结构，能自主执行构建和测试，能在出错时自我修复，能记住我的编码习惯和项目上下文，而且所有数据都在本地，安全可控。

飞虹 Code 就是在这样的背景下诞生的。从 2026 年 8 月写下第一行代码开始，我利用业余时间持续迭代，先后经历了 v7.2 基础架构、v7.5 企业级能力、v7.6 SWE Agent、v7.9 Hermes 自我进化框架，到今天的 v8.0 版本。v8.0 是一个里程碑式的版本，我花了大量精力重构数据存储层（基于 Node.js 内置的 node:sqlite 实现统一存储，零外部依赖）、增加 Docker 沙盒执行能力（解决 AI 生成代码的安全隔离问题）、集成 Honcho 语义记忆（实现真正的跨会话用户画像）、新增免密网络层（让用户安装后零配置即可使用）、打通微信/飞书/元宝多平台消息桥接。每一个功能都是我在实际使用中发现问题、设计方案、编码实现、反复测试后加入的。

开发飞虹 Code 的过程本身也是一次奇妙的体验——我用飞虹 Code 来开发飞虹 Code。代理帮我写代码、做评审、跑测试、修 Bug，我则专注于架构设计和产品决策。这种“人机协作”的开发模式让我深刻认识到，AI 不是要取代开发者，而是要成为开发者最得力的伙伴。飞虹 Code 的核心理念就是让 AI 成为真正的编程伙伴，而非简单的代码补全工具。

这本使用手册记录了飞虹 Code v8.0 的全部功能和使用方法。希望它能帮助你快速上手，也期待你在使用过程中提出宝贵的意见和建议。飞虹 Code 会持续迭代，不断进化。

吴赐虹

2026 年 9 月 3 日 于福建泉州

第一章 软件概述

1.1 产品简介

飞虹 Code（英文名称 fhcode）是一款面向终端开发者的 AI 编程智能体软件，由吴赐虹独立开发完成。软件的开发灵感来源于开发者在实际工作中的真实痛点：传统的代码补全工具只能提供单行或单函数级别的建议，无法理解整个项目的架构和上下文；在线 AI 对话工具无法访问本地代码库，不能执行命令验证结果，数据安全也无法保障。飞虹 Code 正是为了解决这些问题而设计的，它对标国际主流的 Muse Code 和 Cursor CLI 产品，通过大语言模型驱动的自主代理架构，实现软件开发全流程的自动化和智能化。

软件支持多模型路由、全自动软件工程代理、企业级安全审计、代码评审与重构、技能系统与自进化学习、持久化记忆、Docker 沙盒执行、SQLite 统一数据存储、多平台消息桥接等核心能力，同时支持完全离线的私有化部署模式和移动端 Android APP。与市面上其他 AI 编程工具相比，飞虹 Code 的差异化特点在于：第一，真正的终端原生体验，所有操作都在开发者熟悉的命令行环境中完成，无需切换到图形界面；第二，完整的自主代理闭环，从需求理解到代码实现、从构建验证到错误修复，代理能够自主完成全流程，无需人工逐步引导；第三，数据完全本地化，所有代码、记忆、配置都存储在用户本地，模型 API Key 自动加密，支持完全离线使用本地模型，满足企业数据安全合规要求；第四，高度可扩展，支持自定义模型、自定义技能、MCP 外部工具、多平台消息桥接，能够融入开发者现有的工具链。

软件采用 TypeScript 和 Node.js 技术栈开发，具备良好的跨平台兼容性，支持 Windows 10/11（64位）、macOS 11 及以上版本（含 Apple Silicon 芯片）以及主流 Linux 发行版（Ubuntu 20.04+、CentOS 8+、Debian 10+ 等）。软件通过命令行界面（CLI）、Web 控制台、Electron 桌面端和 Android APP 四入口提供服务，开发者既可以在熟悉的终端环境中进行交互式编程对话，也可以通过浏览器访问 Web 控制台进行任务管理、状态监控和日志审计，还可以在手机端通过 APP 随时随地进行对话、创作和游戏。

软件内置灵活的模型路由层，支持接入多种主流大语言模型服务，包括但不限于 DeepSeek、通义千问、Agnes、Ollama 本地模型以及任意兼容 OpenAI API 协议的第三方模型服务。v8.0 版本新增免密网络层，内置五家厂商的免费模型接口，采用环形轮转（Ring Rotation）和环形故障转移（Ring Failover）机制，用户安装后无需配置任何 API Key 即可直接开始对话。当某个模型服务出现故障或限流时，路由层会自动切换到下一个可用模型，保证服务的连续性和稳定性。

飞虹 Code 的核心理念是让 AI 成为真正的编程伙伴，而非简单的代码补全工具。与传统的代码补全插件不同，飞虹 Code 通过自主代理架构，能够理解复杂的软件开发目标，自动将大目标拆解为可执行的子任务序列，调用文件读写、代码搜索、Shell 执行、构建验证、测试运行、Docker 沙

盒等多种工具，完成从需求分析到代码实现、从测试验证到文档生成的完整开发闭环。软件还具备强大的自我修复能力，当构建失败或测试不通过时，代理能够自动分析错误原因并迭代修复，直至任务成功完成。

1.2 技术架构

飞虹 Code 采用分层模块化的软件架构设计，自下而上分为运行时层、存储层、工具层、模型层、代理层、技能层、桥接层和表现层八个层次。各层之间通过清晰的接口进行通信，具备良好的可扩展性和可维护性。

运行时层提供软件运行所需的基础能力，包括 Git 工作区管理、会话持久化存储、事件日志系统、钩子机制（SessionStart/PostToolUse/PreCompact）、工作树管理等。存储层是 v8.0 新增的核心层次，基于 Node.js 内置的 node:sqlite 模块实现统一数据存储，包含十张数据表（元数据、键值存储、任务队列、模型配置、代理配置、技能管理、记忆存储、记忆历史、用户画像、用户记忆、知识库），支持模型 API Key 的 AES-256-GCM 加密落盘和版本化迁移机制，零外部依赖。工具层封装了代理可调用的各种工具，包括文件操作工具、代码搜索工具、Shell 执行工具、Docker 沙盒执行工具、构建验证工具、Web 浏览工具、MCP 客户端工具等。模型层实现了多模型供应商适配、自动择优路由、异常轮换与退避重试、免密网络层等能力。

代理层是软件的核心调度引擎，包含编排器、规划器、代码生成器、质量门控、自我修复、上下文压缩、经验管理、子代理、SWE Agent、多 Agent 协作、事件驱动 Agent、自定义 Agent 等核心组件。编排器采用思考-行动-观察循环模式驱动代理执行任务，接收用户目标后调用规划器进行任务拆解，然后依次调用模型生成工具调用、执行工具并观察结果，循环迭代直至任务完成或达到最大轮次限制。技能层提供可扩展的技能框架，兼容 agentskills.io 开放标准，支持技能的自动提炼、搜索、安装和管理，内置内容摘要、中英互译、代码审查、小红书文案等技能。桥接层是 v8.0 新增的多平台消息网关，支持微信（企业微信/公众号）、飞书、元宝、GitHub MCP Server 等平台的接入，实现一套 Agent 多入口交互。表现层包括 CLI 交互界面、Web 控制台、Electron 桌面端和 Android APP。

1.3 应用场景

飞虹 Code 适用于多种软件开发和运维场景。在日常编码场景中，开发者可以通过自然语言描述开发需求，代理自动生成代码文件、修改现有代码、执行搜索替换、添加依赖配置，大幅提升编码效率。例如，开发者输入“给用户模块增加密码重置功能，需要生成重置令牌、发送重置邮件、验证令牌后更新密码”，代理会自动分析现有用户模块的代码结构，生成令牌生成和验证工具、邮件发送服务、密码重置 API 路由，并更新相关的数据库表结构和前端页面。

在代码评审场景中，软件内置结构化代码评审引擎，能够扫描代码库中的潜在问题，包括安全漏洞、性能瓶颈、代码规范违规、复杂度过高、可维护性差等，按严重程度分为四个等级输出评审报告和修复建议。在自动化测试场景中，软件能够根据代码变更自动生成单元测试用例，执行测试并分析覆盖率，当测试失败时自动修复测试代码或被测代码。在遗留系统重构场景中，代理能够理解现有代码结构，制定渐进式重构计划，逐步执行代码迁移、架构调整、依赖升级，同时保证构建和测试持续通过。在 DevOps 场景中，软件支持通过 Webhook 触发自动化任务，与 CI/CD 流水线深度集成，实现代码提交后的自动评审、自动测试、自动部署全流程。在企业级应用场景中，飞虹 Code 提供多租户隔离、基于角色的访问控制、操作审计日志、资源配额管理等企业级能力。

在移动端场景中，用户可以通过 Android APP 随时随地进行 AI 对话、文生图、文生视频、图生视频创作，享受内置的十二款小游戏，配置自定义大模型接口进行图片识别理解。在知识管理场景中，软件的持久记忆系统和 Honcho 语义记忆能够自动记录用户偏好、项目上下文和历史任务，跨会话检索相关记忆，无需每次重新交代背景。在自动化调度场景中，用户可以通过自然语言设定定时任务（每天/每周/间隔/单次），替代手写 cron，实现定时备份、定时报表、定时巡检等自动化工作流。

1.4 系统要求

飞虹 Code 的运行环境要求如下。操作系统方面，支持 Windows 10 版本 21H2 及以上（64 位）、Windows 11 全部版本、macOS 11 Big Sur 及以上版本（支持 Intel 和 Apple Silicon 芯片）、Linux 内核版本 4.15 及以上的主流发行版。运行时环境方面，需要 Node.js 22.0.0 及以上版本（v8.0 版本使用 Node.js 内置的 node:sqlite 模块，要求 Node.js 22+），推荐使用 Node.js 22 LTS 版本以获得最佳性能和稳定性。硬件资源方面，最低配置要求 4GB 系统内存、1GB 可用磁盘空间、x86_64 或 ARM64 架构处理器。推荐配置为 8GB 以上系统内存、2GB 可用磁盘空间、多核处理器。对于使用 Ollama 本地模型的用户，推荐至少 8GB 显存的 GPU。

可选组件方面，Docker 沙盒执行功能需要 Docker 20.10 及以上版本（推荐 Docker Desktop 29.7.2），用于在隔离容器中执行代码和命令。Android APP 需要 Android 8.0 及以上版本的移动设备。Web 控制台支持 Chrome、Firefox、Safari、Edge 等主流现代浏览器。

第二章 安装部署

2.1 npm 安装

飞虹 Code 已发布到 npm 公共软件仓库，用户可以通过 npm 命令进行全局安装，这是最推荐的安装方式。在安装前，请确保系统已安装 Node.js 22.0.0 及以上版本，可以在终端中执行 `node -v` 命令验证 Node.js 版本。在 Windows 系统中，打开命令提示符或 PowerShell 终端，执行 `npm install -g feihong-code` 命令进行全局安装。安装完成后执行 `fhcode --version` 命令验证安装是否成功，预期输出 8.0.0。如果遇到权限错误，可以尝试以管理员身份运行终端后重新执行安装命令。

在 macOS 和 Linux 系统中，打开终端应用，执行 `sudo npm install -g feihong-code` 命令进行全局安装。如果遇到 EACCES 权限错误，除了使用 `sudo` 外，还可以修改 npm 全局安装目录到用户主目录下。软件的升级和卸载同样通过 npm 命令完成：升级执行 `npm update -g feihong-code`，查看版本执行 `npm list -g feihong-code`，卸载执行 `npm uninstall -g feihong-code`。安装完成后，首次运行 `fhcode` 命令会自动在用户主目录下创建 `.feihong-code` 配置目录和 `feihong.db` SQLite 数据库文件。

2.2 Docker 部署

飞虹 Code 提供官方 Docker 镜像，支持容器化部署，特别适合企业级私有化部署和 CI/CD 环境。使用 Docker 部署可以避免环境依赖问题，保证运行环境的一致性，同时便于快速扩展和迁移。在部署前，请确保系统已安装 Docker 20.10 及以上版本和 Docker Compose v2。使用 `docker run` 命令快速启动单容器实例：`docker run -d --name fhcode -p 8080:8080 -e FH_WEB_TOKEN=your-token feihong-code:8.0.0`。启动后通过浏览器访问 `http://localhost:8080` 即可打开 Web 控制台。

推荐使用 `docker-compose.yml` 进行部署，便于管理配置和数据持久化。创建 `docker-compose.yml` 文件，配置服务名称、镜像版本、端口映射、环境变量、数据卷挂载等选项。环境变量中配置 `FH_PROVIDERS`（模型提供商 JSON 数组）、`FH_WEB_PORT`（Web 控制台端口）、`FH_WEB_TOKEN`（Web 控制台访问令牌）、`FH_DB_PATH`（SQLite 数据库路径）、`FH_HONCHO_USER`（Honcho 当前用户 ID）等。`volumes` 中挂载配置目录和数据目录到宿主机，实现数据持久化。配置完成后执行 `docker compose up -d` 启动服务，`docker compose logs -f` 查看实时日志，`docker compose down` 停止并移除服务。企业级部署建议通过 Nginx 或 Traefik 反向代理提供 HTTPS 访问，配置 SSL 证书。

2.3 源码编译安装

对于希望从源代码编译安装的用户，可以从 GitHub 仓库克隆源代码后进行本地构建。这种方式适合开发者参与项目贡献、自定义修改或使用最新开发版本。在编译前，请确保系统已安装 Git、Node.js 22+ 和 npm。克隆源代码：执行 `git clone https://github.com/wch887292/feihong-code.git` 命令克隆仓库到本地，然后 `cd feihong-code` 进入项目目录。安装依赖：执行 `npm install` 命令安装项目依赖。编译构建：执行 `npm run build` 命令进行 TypeScript 编译和 Web 静态资源复制。构建脚本会先执行 `tsc` 命令将 TypeScript 源码编译为 JavaScript，输出到 `dist` 目录，然后执行 `node scripts/copy-web.cjs` 脚本将 Web 控制台的静态资源复制到 `dist/web/public` 目录。

本地运行：执行 `npm start` 或 `node dist/cli/index.js` 命令启动软件。全局链接：执行 `npm link` 命令将本地构建的软件链接到全局 npm 目录，之后就可以在任意目录使用 `fhcode` 命令，适合开发调试。运行测试：执行 `node tests/smoke-v8.js` 命令运行 v8.0 冒烟测试套件（25 项），执行 `node tests/integration/api.test.js` 命令运行 API 集成测试套件（34 项），验证代码功能正确性。

2.4 模型配置

模型配置是飞虹 Code 正常运行的关键步骤。v8.0 版本支持两种模式：免密网络层模式和自定义模型模式。

免密网络层模式：用户无需配置任何 API Key，安装后直接使用。软件内置五家厂商的免费模型接口，采用环形轮转和环形故障转移机制。当某个模型服务出现故障或限流时，自动切换到下一个可用模型。网页搜索同样开箱即用，内置 Brave、Tavily、Firecrawl 等搜索工具的环形轮转，无需注册任何 API Key。

自定义模型模式：通过 `FH_PROVIDERS` 环境变量配置一个或多个模型提供商。`FH_PROVIDERS` 为 JSON 数组格式，每个数组元素描述一个模型提供商的完整配置信息，包括 `id`（唯一标识）、`name`（显示名称）、`provider`（类型：`openai-compatible` 或 `ollama`）、`apiBase`（API 基础地址）、`apiKey`（API 访问密钥）、`model`（模型 ID）、`contextWindow`（上下文窗口大小）、`maxOutput`（最大输出 token 数）、`supportsStreaming`（是否支持流式输出）、`supportsVision`（是否支持视觉/图片理解）等字段。配置示例包含 Agnes-2.5-Flash（地址 `https://api.agnes-ai.cn/v1`）、DeepSeek-V3（地址 `https://api.deepseek.com/v1`）、本地 Ollama（地址 `http://localhost:11434/v1`）三个提供商。视觉模型（如 DeepSeek-OCR、GPT-4o）需要将 `supportsVision` 设置为 `true`，用于图片上传后的识别理解。

配置完成后，软件会根据默认的成本优先策略自动选择最优模型。用户可以通过 `--strategy` 命令行参数临时切换策略（`cost/capability/latency`），或在 Web 控制台的模型管理页面手动指定当

前任务使用的模型。模型 API Key 在 SQLite 数据库中自动 AES-256-GCM 加密落盘，主密钥来自 FH_SECRET 环境变量或自动生成的 .secret 文件，确保敏感数据安全。

2.5 环境验证

安装和配置完成后，建议进行环境验证以确保软件正常运行。版本验证：执行 `fhcode --version` 命令，确认输出版本号为 8.0.0。SQLite 存储验证：启动服务后访问 `http://localhost:8080/api/health`，确认返回 JSON 中 `storage.ok` 为 `true`、`backend` 为 `sqlite`。Honcho 记忆验证：访问 `/api/health` 确认 `honcho.ok` 为 `true`。Docker 沙盒验证：执行 `docker info` 确认 Docker daemon 正常运行，执行 `node tests/smoke-v8.js --only=docker` 运行 Docker 沙盒冒烟测试（6 项）。免密模式验证：在未配置 FH_PROVIDERS 的情况下，执行 `fhcode "用一句话介绍你自己"` 命令，观察代理是否能够通过免密网络层调用免费模型生成回复。Web 控制台验证：执行 `fhcode serve` 命令启动 Web 控制台，在浏览器中访问 `http://localhost:8080`，创建一个测试任务观察执行情况。

第三章 快速上手

3.1 首次启动

安装完成后，在终端中执行 `fhcode` 命令即可启动飞虹 Code 的交互式对话界面。首次启动时，软件会进行初始化操作，包括在用户主目录下创建 `.feihong-code` 配置目录、生成默认配置文件和主密钥（`.secret`）、初始化 SQLite 数据库（`feihong.db`，自动创建十张数据表）、初始化会话存储、加载内置技能、初始化 Honcho 语义记忆用户画像等。初始化完成后，终端会显示欢迎信息，包括软件名称、版本号（8.0.0）、当前模式（在线/离线/免密）、配置的模型数量、SQLite 存储状态、Honcho 记忆状态等信息，并提示用户输入开发需求。如果未配置 `FH_PROVIDERS` 环境变量，软件自动进入免密网络层模式，使用内置的五家免费模型环形轮转，无需任何 API Key。

启动交互式对话后，用户可以直接输入自然语言描述开发需求，例如“创建一个 Python 的 FastAPI 服务器，包含用户注册登录接口”。代理接收到目标后，会自动进行任务规划，将大目标拆解为可执行的子任务序列，然后依次执行。执行过程中，终端会实时显示当前执行步骤、调用的工具名称、工具输入参数、工具执行结果、代理思考过程等信息。任务完成后，代理会输出总结报告，包括完成的子任务列表、生成或修改的文件清单、构建和测试验证结果、耗时统计等信息。用户可以继续输入新的需求进行迭代开发，软件支持上下文记忆和 Honcho 语义记忆，跨会话保留用户偏好和项目上下文。

3.2 交互式对话操作

在交互式对话界面中，用户可以通过自然语言与代理进行多轮对话协作。基本操作包括：输入需求后按回车键发送，代理开始执行；执行过程中发送按钮会变为停止按钮，点击可中断当前执行；使用上下方向键浏览历史输入记录；使用 `Tab` 键自动补全文件名和命令。对话过程中支持多种斜杠命令，用于快速切换模式或触发特定功能：`/plan` 切换到规划模式，代理仅输出任务规划方案而不执行；`/grill` 对当前工作区的代码进行深度拷问式审查；`/review 路径` 对指定目录或文件进行结构化代码评审；`/goal` 启动目标驱动模式；`/memory` 查看当前记忆状态（短期记忆+长期记忆+Honcho 用户画像）；`/skills` 列出已安装技能；`/models` 列出已配置模型；`/status` 查看 Agent 状态（包括 SQLite 存储和 Docker 沙盒状态）；`/clear` 清空当前对话上下文；`/exit` 或 `/quit` 退出对话界面；`/help` 查看所有可用的斜杠命令。

多轮对话上下文管理：软件自动维护对话上下文，包括历史消息、已执行的工具调用、文件变更记录等。代理在后续对话中能够引用之前的操作结果，实现连贯的迭代开发。当对话过长导致上下文超出模型限制时，软件自动触发上下文压缩（PreCompact 钩子），将历史对话摘要为紧凑形式，保留关键信息的同时减少 token 占用。Honcho 语义记忆系统会自动从用户消息中提取偏好类

事实，沉淀到用户画像和事实记忆中，跨会话检索相关记忆注入上下文，实现个性化的对话体验。

输出格式说明：代理的输出分为思考区域、工具调用区域、工具结果区域、最终回复区域。工具调用区域使用不同的颜色区分不同类型的工具，便于用户快速识别。

3.3 命令行参数详解

飞虹 Code 提供丰富的命令行参数，支持灵活配置运行行为。全局参数（适用于所有子命令）包括：--version 或 -v 显示版本号并退出；--help 或 -h 显示帮助信息并退出；--offline 强制使用离线模式；--strategy 指定模型选择策略（cost/capability/latency）；--model 指定使用的模型名称；--max-cycles 指定最大执行轮次（默认 50）；--config 指定配置文件路径；--log-level 指定日志级别（debug/info/warn/error）。fhcode chat 子命令参数包括 --goal 启动时直接执行指定目标、--context-file 注入上下文文件、--no-welcome 不显示欢迎信息。

fhcode run 子命令参数包括 goal（位置参数，必填）要执行的任务目标、--json 输出 JSON 格式结果、--output 将结果输出到指定文件、--timeout 任务超时时间。fhcode review 子命令参数包括 path（位置参数，可选）要评审的目录或文件路径、--json 输出 JSON 格式、--threshold 阻断阈值、--rules 启用规则列表、--exclude 排除文件模式。fhcode serve 子命令（启动 Web 控制台）参数包括 --port 指定监听端口（默认 8080）、--host 指定监听地址、--token 指定访问令牌、--no-browser 启动后不自动打开浏览器。

3.4 免密网络层使用指南

免密网络层是 v8.0 的核心特色功能，用户安装后无需配置任何 API Key 即可直接开始对话和搜索。免密网络层内置五家厂商的免费模型接口，采用环形轮转（Ring Rotation）机制，每次调用按顺序选择下一个模型，实现负载均衡。当某个模型服务出现故障、限流或超时，触发环形故障转移（Ring Failover），自动切换到下一个可用模型，无需人工干预。网页搜索同样开箱即用，内置 Brave、Tavily、Firecrawl 等搜索工具的环形轮转，用户无需去三个平台注册账号即可使用搜索功能。

免密网络层的核心价值在于：零配置上手，安装后直接开聊，降低使用门槛；高可用性，多模型环形故障转移，单点故障不影响整体服务；成本为零，使用免费模型接口，无需支付 API 费用。当用户需要更高质量的模型或特定能力时，可以随时配置自定义模型（FH_PROVIDERS），软件会优先使用自定义模型，免密网络层作为后备。从免密模式切换到自定义模型模式：设置 FH_PROVIDERS 环境变量并配置至少一个有效模型提供商，然后重启软件或启动新会话即可使用自定义大模型。

3.5 移动端 APP 使用

飞虹 Code 提供 Android APP，用户可以在手机端随时随地使用 AI 能力。APP 安装：将 APK 文件安装到 Android 手机（要求 Android 8.0+），打开后即可使用。APP 主界面采用底部导航栏布局，包含对话、游戏中心、创作、技能、我的五个模块。对话模块：支持多轮对话、流式输出，发送框支持添加图片、上传文件、截图、拍照，发送时发送按钮变为停止按钮可中断生成。游戏中心：内置十二款小游戏（包括益智类、休闲类、动作类等），点击即玩，适配手机屏幕尺寸和触摸操作。创作模块：支持文生图（输入描述生成图片）、文生视频（输入描述生成视频）、图生视频（上传图片生成视频），支持自定义配置默认模型。技能模块：内置内容摘要、中英互译、小红书文案等技能，点击即可使用。我的模块：自定义大模型配置（支持配置任意 OpenAI 兼容模型接口，包括视觉模型）、Honcho Agent 持久记忆查看、免密网络层状态、关于等。

第四章 核心功能详解

4.1 多模型路由

多模型路由是飞虹 Code 的核心能力之一，允许用户同时配置多个模型提供商，软件根据预设策略自动选择最优模型进行调用。支持的模型选择策略包括三种：cost（成本优先）选择每千 token 成本最低的可用模型，适合预算敏感场景；capability（能力优先）根据任务类型匹配具备相应能力标签的模型，例如代码生成任务优先选择带有 code-gen 标签的模型，图片理解任务优先选择支持视觉的模型；latency（延迟优先）选择历史平均响应时间最短的模型，适合交互体验敏感的场景。路由层维护每个模型的实时性能统计数据，包括总调用次数、成功调用次数、失败调用次数、累计成本、平均延迟、最后使用时间、成功率等关键指标。统计数据持久化到 SQLite 数据库的 models 表中。

异常处理和自动轮换机制是路由层的重要组成部分。当上游模型返回错误时，路由层根据错误类型采取不同的处理策略：对于 400/401/403/404 等鉴权或模型类错误，立即轮换到下一个可用模型，不重试同一模型；对于 429/500/502/503/504 等限流或服务端瞬时错误，轮换到下一个可用模型并进行指数退避后重试整轮，直至任务完成或达到最大重试次数；对于网络连接错误、超时、DNS 解析失败等网络类错误，按可重试错误处理，同样触发模型轮换和退避重试。免密网络层在此基础上增加了环形故障转移机制，五家免费模型环形轮转，单点故障自动切换。模型路由层支持运行时动态添加和移除模型提供商，无需重启软件，用户可以通过 Web 控制台的模型管理页面进行管理。

4.2 SWE Agent 全自动软件工程代理

SWE Agent（Software Engineering Agent）是飞虹 Code 的高级代理模式，专为复杂软件工程任务设计。与普通对话模式不同，SWE Agent 采用规划-执行-验证-修复的完整闭环架构，能够自主处理从需求理解到代码实现、测试验证的全流程。SWE Agent 特别适合处理涉及多文件修改、架构调整、依赖升级、Bug 修复、功能开发等复杂开发任务，能够显著减少人工干预，提升开发效率和代码质量。SWE Agent 的执行流程分为四个阶段：规划阶段由 SWE 规划器接收用户目标进行深度分析，生成结构化的执行计划，包括任务拆解、文件变更预估、风险评估、验证策略等；执行阶段代理按照规划的子任务序列逐步执行，调用文件操作、代码搜索、Shell 执行、构建验证、Docker 沙盒等工具。

验证阶段由 SWE 验证器在执行过程中持续监控构建状态和测试结果，每当代理完成可能影响构建或测试的操作后，验证器自动触发构建检查和测试运行，确认代码变更没有引入新的错误。自我修复阶段当工具执行失败或验证不通过时自动触发，对错误进行深度分析和迭代修复。自我修复模

块首先对错误进行分类（文件不存在、构建错误、命令未找到、参数无效、测试失败、依赖冲突、权限不足、未知错误等），然后根据错误类型生成针对性的反思提示，注入到代理的上下文中，引导模型分析错误的根本原因并制定修复方案。代理根据反思提示重新规划修复步骤，执行修复操作后再次验证，循环迭代直至问题解决或达到最大修复轮次。v8.0 版本在 SWE Agent 中集成了源文件定位率提升方案、no_patch 分析与 prompt 优化、多模型对比与路由、基于强化学习和贝叶斯优化的自动调参路由策略，持续提升 SWE-bench 评测的 resolved-rate。

4.3 持久记忆系统

持久记忆系统是飞虹 Code 的核心能力之一，实现跨会话的长期记忆，无需用户每次重新交代背景。记忆系统分为四个层次：短期记忆存储当前会话的上下文，包括历史消息、工具调用、文件变更等，自动压缩管理；长期记忆以 MEMORY.md 文件形式存储项目上下文、关键事实、历史任务摘要，自动更新；用户画像以 USER.md 文件形式存储用户偏好、习惯、专业领域，跨项目共享；Honcho 语义记忆（v8.0 新增）基于 SQLite 数据库，实现用户建模、事实记忆、会话摘要、跨会话语义检索，支持重要度排序和记忆注入提示词。

记忆自动沉淀机制：每轮对话结束后，软件自动生成对话摘要，更新 MEMORY.md；从用户消息中自动提取偏好类事实，沉淀到 Honcho 用户画像和事实记忆中，附带重要度评分（0~1）；关键决策和技术方案自动记录到项目经验库。记忆检索与注入：当代理接收新任务时，记忆系统自动根据任务目标检索相关记忆，包括项目上下文、历史类似任务、用户偏好等，以结构化提示的形式注入到代理的上下文中。Honcho 语义记忆支持关键词匹配+重要度排序+时间加权的检索算法，返回最相关的事实记忆和会话摘要。记忆管理：用户可以通过 /memory 命令查看当前记忆状态，通过 Web 控制台的记忆中心管理记忆内容，手动添加、编辑、删除记忆条目。记忆数据默认存储在本机（SQLite 数据库和 Markdown 文件），用户可以自主控制数据的导出和共享，也可以配置 FH_HONCHO_URL 指向外部 Honcho 服务实现云端记忆。

4.4 技能系统与自演化

技能系统是飞虹 Code 的可扩展能力框架，允许用户为代理添加特定领域的专业技能，使代理能够处理特定类型的任务。每个技能是一个独立的配置文件（YAML 或 Markdown 格式），描述技能的元数据（名称、描述、版本、作者、标签）、触发条件（在什么情况下自动激活该技能）、执行步骤（技能执行的具体流程，包括提示模板、工具调用、条件判断、循环等）、输入输出规范等。代理在执行任务时，根据目标描述和当前上下文自动匹配并加载相关技能，按照技能定义的流程执行，从而获得特定领域的专业能力。软件兼容 agentskills.io 开放标准，支持从技能市场搜索和安装社区贡献的技能包。内置技能包括内容摘要、中英互译、代码审查、小红书文案生成等。v7.9 版本新增 LLM 辅助技能提炼，从重复任务模式中自动提炼可复用技能，提升提取准确率。

自进化是飞虹 Code 的高级特性，使代理能够在执行任务过程中自动积累经验，从成功和失败中学习，持续提升执行能力和效率。自进化系统的核心是经验管理模块，该模块记录每次任务执行的完整轨迹，包括任务目标、规划方案、工具调用序列、执行结果、验证状态、最终成败、耗时统计、成本统计等关键信息。经验数据以结构化格式持久化存储到 SQLite 数据库，支持高效检索和分析。当代理遇到类似任务时，可以检索历史经验，借鉴成功的执行模式和工具调用序列，避免重复之前的错误，从而提升首次成功率和执行效率。自进化闭环：飞虹 Code 的自进化系统形成完整的执行-记录-检索-应用-优化闭环。代理执行任务时自动积累经验，后续任务参考历史经验提升表现，新的执行结果又反馈到经验库中，持续丰富和优化经验数据。需要注意的是，自进化系统仅记录执行轨迹和元数据，不记录敏感信息，经验数据默认存储在本地，用户可以自主控制数据的导出和共享。

4.5 自动化调度

自动化调度功能允许用户通过自然语言设定定时任务，替代手写 cron，实现定时备份、定时报表、定时巡检、定时推送等自动化工作流。用户只需用自然语言描述任务的执行时间和内容，软件自动解析为调度规则并执行。支持的自然语言时间表达包括：每天 9:00（daily，每天指定时间执行）、每周一 10:30（weekly，每周指定星期和时间执行）、每隔 30 分钟（interval，按固定间隔执行）、14:00（once，今天指定时间执行一次）。调度任务创建后，软件在后台持续监控，到达指定时间自动触发任务执行，执行结果记录到任务队列和审计日志。

自动化调度的典型应用场景包括：定时备份（每天凌晨自动备份项目代码和数据库）、定时报表（每周一自动生成上周的项目进度报表和代码质量报告）、定时巡检（每小时自动检查服务健康状况、磁盘空间、日志异常）、定时推送（每天早上自动推送天气、新闻、待办事项）、定时测试（每天夜间自动运行全量测试套件，第二天早上查看结果）。调度任务支持任务队列管理，用户可以在 Web 控制台查看所有定时任务的列表、执行历史、下次执行时间，支持手动触发、暂停、恢复、删除定时任务。

第五章 v8.0 新增功能

5.1 SQLite 统一数据存储

SQLite 统一数据存储是 v8.0 的核心新增功能，基于 Node.js 内置的 node:sqlite 模块实现，零外部依赖。解决了原文件/JSON 存储在大规模数据时性能下降、查询困难、模型 API Key 明文存储有安全风险等问题。

核心能力包括：十张数据表统一管理（meta 元数据、kv 键值存储、tasks 任务队列、models 模型配置、agents 代理配置、skills 技能管理、memory 记忆存储、memory_history 记忆历史、users 用户画像、user_memory 用户事实记忆、knowledge 知识库），覆盖软件运行所需的全部数据类型；模型 API Key 自动 AES-256-GCM 加密落盘，主密钥来自 FH_SECRET 环境变量或自动生成的 .secret 文件，确保敏感数据安全；版本化迁移机制（SCHEMA_VERSION），启动时自动检查数据库版本，执行增量迁移，支持平滑升级；WAL 日志模式提升并发读写性能，外键约束保证数据完整性。

使用方式：无需额外配置，启动后自动在 FH_HOME 目录下创建 feihong.db 数据库文件（可通过 FH_DB_PATH 环境变量自定义路径）。首次启动自动创建十张数据表和索引。API 接口：GET /api/storage/stats 查看存储统计（各表记录数、数据库大小、版本号）；模型配置通过 GET/POST/PUT/DELETE /api/models 接口管理，API Key 自动加密存储。数据备份：直接复制 feihong.db 文件即可完成全量备份，支持 SQLite 标准工具进行数据导出和导入。

5.2 Docker 沙盒执行

Docker 沙盒执行是 v8.0 的核心安全功能，在 Docker 容器中隔离执行代码和命令，降低 AI 生成代码直接在宿主机执行的安全风险。解决了直接在宿主机执行 AI 生成的代码有安全风险、不同项目依赖冲突等问题。

安全机制包括：网络隔离（默认禁用网络，--network none，容器内无法访问外部网络，防止数据外泄和恶意网络请求）；只读挂载（工作区默认只读挂载到容器，防止容器内进程修改宿主机文件，可通过 FH_SANDBOX_READONLY=false 关闭）；资源限制（默认内存限制 512MB -m 512m，CPU 限制 1 核 --cpus 1，防止容器耗尽宿主机资源）；危险命令拦截（在容器启动前扫描命令，拒绝 rm -rf /、mkfs、dd 写块设备、fork 炸弹等危险命令）；超时强杀（默认超时 60 秒，超时后自动执行 docker kill 强制终止容器，防止无限循环或挂起进程）。

使用方式：需要本机安装 Docker Desktop 或 Docker Engine (20.10+)。环境变量配置：FH_SANDBOX_IMAGE 指定沙盒镜像（默认 node:22-alpine）、FH_SANDBOX_NETWORK 是否

启用网络（默认 false）、FH_SANDBOX_READONLY 是否只读挂载（默认 true）、FH_SANDBOX_TIMEOUT 超时时间毫秒（默认 60000）。可用性检测：软件启动时自动检测 Docker 可用性（通过 docker info 命令确认 daemon 在运行，而非仅检查 CLI 存在），不可用时沙盒功能自动降级为本地执行并记录警告。冒烟测试：node tests/smoke-v8.js --only=docker 运行六项 Docker 沙盒测试（可用性检测、容器创建、命令执行、网络隔离、资源限制、超时强杀）。国内镜像加速器：Docker Hub 在国内无法直接访问时，配置 daemon.json 注册国内镜像加速器（docker.1ms.run、docker.m.daocloud.io、hub-mirror.c.163.com、mirror.baidubce.com），注意 daemon.json 必须保存为无 BOM 的 UTF-8 编码，否则 Docker 解析失败。

5.3 Honcho 语义记忆

Honcho 语义记忆是 v8.0 新增的用户建模和语义记忆系统，对标 Honcho (GetZep) 的语义记忆能力，实现跨会话的用户画像、事实记忆和语义检索。解决了原文件记忆只能按文件读写、无法按用户/重要度/关键词检索、跨设备跨会话的用户画像和偏好无法持久化等问题。

核心能力包括：用户建模（users 表存储用户姓名、偏好、特质画像，支持多用户隔离，通过 FH_HONCHO_USER 指定当前用户）；事实记忆（user_memory 表存储用户说过的关键事实，附带重要度评分 0~1，支持按重要度排序和时间衰减）；会话摘要（memory_history 表存储每轮对话的自动摘要，支持跨会话检索历史上下文）；语义检索（关键词匹配+重要度排序+时间加权的检索算法，返回最相关的事实记忆和会话摘要）；记忆注入（自动生成个性化上下文提示词 buildContextPrompt，包含用户画像、相关事实记忆、近期会话摘要，注入到每次对话的系统提示中）；自动沉淀（digestConversation 从用户消息中自动提取偏好类事实，附带重要度评分，沉淀到用户画像和事实记忆中）。

使用方式：无需额外配置，启动后自动启用（SQLite 本地持久化，复用 sqlite-store）。API 接口：GET /api/honcho/context 获取当前用户画像 + 记忆 + 上下文提示词；POST /api/honcho/remember 记录用户事实记忆（参数：content 内容、importance 重要度 0~1）；GET /api/honcho/recall?q=关键词 关键词检索记忆。环境变量：FH_HONCHO_USER 当前用户 ID（默认 default）、FH_HONCHO_URL 外部 Honcho 服务 URL（预留，配置后走远程模式）、FH_HONCHO_API_KEY 外部 Honcho API Key（预留）。默认本地部署模式下所有记忆数据存储在本地 SQLite 数据库中，用户可以自主控制数据；配置外部 Honcho 服务后可实现云端记忆和多设备同步。

5.4 多平台消息桥接

多平台消息桥接是 v8.0 新增的多入口交互能力，实现一套 Agent 多平台消息网关，用户可以在微信、飞书、元宝等平台与飞虹 Code Agent 交互，一处发起任务，另一处继续对话。

微信桥接：支持企业微信（wecom）和公众号（mp）两种模式。企业微信模式通过 FH_WECHAT_MODE=wecom 启用，配置 FH_WECHAT_CORP_ID（企业 ID）、FH_WECHAT_AGENT_ID（应用 ID）、FH_WECHAT_SECRET（应用密钥）、FH_WECHAT_TOKEN（回调 Token）、FH_WECHAT_AES_KEY（消息加密密钥），回调地址为 https://域名/api/wechat/callback。公众号模式通过 FH_WECHAT_MODE=mp 启用，配置 FH_WECHAT_APP_ID、FH_WECHAT_APP_SECRET、FH_WECHAT_TOKEN、FH_WECHAT_AES_KEY。用户在微信中发送消息，Agent 自动回复，支持文本、图片、文件等消息类型。

飞书桥接：通过 FH_FEISHU_ENABLED=true 启用，配置 FH_FEISHU_APP_ID（应用 ID）、FH_FEISHU_APP_SECRET（应用密钥）、FH_FEISHU_VERIFICATION_TOKEN（验证 Token）、FH_FEISHU_ENCRYPT_KEY（加密密钥），事件订阅地址为 https://域名/api/feishu/callback。支持飞书机器人单聊和群聊 @机器人 触发，Agent 回复支持富文本、卡片等飞书消息格式。元宝桥接：通过 FH_YUANBAO_ENABLED=true 启用，配置 FH_YUANBAO_APP_ID、FH_YUANBAO_APP_SECRET，实现字节跳动元宝平台的消息接入。GitHub MCP Server：通过 src/integrations/github-mcp.ts 实现，支持连接 GitHub MCP 服务器，代理可以调用 GitHub 相关工具（Issue 管理、PR 管理、代码搜索、仓库操作等），实现与 GitHub 平台的深度集成。

5.5 文生图与文生视频

v8.0 版本新增文生图和文生视频创作能力，用户可以通过自然语言描述生成图片和视频内容，支持在 Web 控制台和移动端 APP 中使用。文生图功能：用户输入图片描述（如“一只在海边奔跑的金毛犬，夕阳背景，油画风格”），选择生成模型，点击生成按钮，软件调用配置的文生图模型 API 生成图片，生成的图片可以下载、分享或作为图生视频的输入。支持自定义配置默认文生图模型，兼容 OpenAI 兼容的图片生成 API 接口。

文生视频功能：用户输入视频描述（如“一个人在雨中漫步，镜头缓慢推进，电影感”），选择生成模型，点击生成按钮，软件调用文生视频模型 API 生成视频，生成过程可能需要较长时间（通常几秒到几分钟），生成完成后可以在线预览和下载。图生视频功能：用户上传一张图片，输入运动描述（如“镜头从左向右缓慢移动”），点击生成按钮，软件基于上传的图片和运动描述生成视频。视频生成支持自定义配置默认模型，在移动端 APP 和 Web 控制台均提供统一的创作界面，界面布局经过优化，退出按钮和功能按钮有明确的间距分隔，避免误触。

5.6 游戏中心

v8.0 版本在移动端 APP 中新增游戏中心，内置十二款小游戏，用户可以在手机上直接打开即玩，无需额外安装。游戏涵盖多种类型：益智类（数独、2048、俄罗斯方块等）、休闲类（消消乐、跳一跳、贪吃蛇等）、动作类（打砖块、飞机大战等）、卡牌类等。每款游戏都针对手机屏幕

尺寸和触摸操作进行了专门适配，游戏画面自适应屏幕大小，操作按钮大小适合手指点击，支持横竖屏切换。游戏中心的设计理念是提供情绪价值，让用户在工作之余可以轻松娱乐，游戏进度本地保存，支持暂停和继续。

第六章 Web 控制台

6.1 控制台概述与启动

飞虹 Code 提供基于浏览器的 Web 控制台，允许用户通过图形界面管理代理任务、监控执行状态、查看审计日志、配置系统参数、管理模型和记忆。Web 控制台采用前后端分离架构，后端基于 Express.js 提供 RESTful API 和 WebSocket 实时推送服务，前端使用原生 HTML/CSS/JavaScript 实现，集成 Monaco Editor 代码编辑器，无需额外的构建工具，轻量高效，加载速度快。控制台支持响应式布局，适配桌面端和移动端浏览器，在手机和平板上也能正常使用。

启动 Web 控制台的方式有多种：命令行方式执行 `fhcode serve` 命令启动 Web 控制台服务，默认监听 8080 端口，可通过 `--port` 参数或 `FH_WEB_PORT` 环境变量修改端口号，`--host` 参数指定监听地址，`--token` 参数指定访问令牌；Docker 方式中 Web 控制台随容器自动启动；系统服务方式中在 Linux 系统中可以配置 `systemd` 服务，设置为开机自启动。访问控制台：在浏览器中输入 `http://服务器地址:端口号` 访问 Web 控制台。如果配置了 `FH_WEB_TOKEN` 环境变量，首次访问会要求输入访问令牌，输入正确后进入控制台主界面。建议在生产环境中配置 `FH_WEB_TOKEN` 启用身份验证，防止未授权访问。企业版部署建议通过 Nginx 或 Traefik 反向代理提供 HTTPS 访问，并配置 SSL 证书，保障数据传输安全。

控制台主界面采用左侧导航栏加右侧内容区的经典布局，左侧导航栏包含对话主页、任务中心、模型配置、技能管理、记忆中心、知识库、插件管理、存储统计、Honcho 记忆、设置、API 文档等功能模块入口，右侧内容区显示当前选中模块的详细内容。控制台的核心技术特性包括：实时推送（通过 WebSocket 实现任务状态变更和执行输出的实时推送）、响应式设计（适配不同屏幕尺寸）、代码编辑器集成（Monaco Editor 支持语法高亮、代码补全、多标签编辑）、数据可视化（任务统计、模型性能、系统资源等数据以图表形式直观展示）、操作审计（所有用户操作均记录审计日志）。

6.2 对话与任务管理

对话主页是 Web 控制台的核心界面，用户可以在浏览器中直接与 Agent 进行多轮对话，支持流式输出、Markdown 渲染、代码块语法高亮、图片上传、文件附件等功能。对话界面采用类似主流 AI 对话产品的布局，左侧为历史会话列表，右侧为当前对话内容区，底部为输入框（支持添加图片、上传文件、截图，发送时发送按钮变为停止按钮可中断生成）。对话上下文自动管理，支持新建会话、切换会话、删除会话、重命名会话。

任务中心管理所有异步任务，用户可以查看任务列表、监控任务执行状态、查看任务详情和完整输出、取消正在执行的任务、删除历史任务记录。任务采用队列机制管理，支持并发执行，默认并发数为 2，可通过 `FH_TASK_CONCURRENCY` 环境变量调整。每个任务拥有唯一的任务 ID（UUID 格式），任务状态包括 `pending`（等待中）、`running`（执行中）、`completed`（已完成）、`failed`（失败）、`cancelled`（已取消）。创建任务：在任务管理页面点击新建任务按钮，弹出任务创建表单，包含任务目标（必填，自然语言描述要完成的开发任务，支持多行输入）、执行模式（可选，普通对话模式/SWE Agent 模式/目标驱动模式）、模型策略（可选，成本优先/能力优先/延迟优先）、指定模型（可选，手动指定使用的模型）、上下文文件（可选，上传文件作为额外上下文注入）、任务标签（可选，为任务添加自定义标签）。任务详情页面展示任务的完整信息，包括任务基本信息、执行步骤时间线、文件变更列表、构建和测试结果、最终总结报告。

6.3 模型与技能管理

模型管理页面展示当前配置的所有模型提供商列表，包括模型 ID、显示名称、类型（OpenAI 兼容/Ollama）、API 基础地址、模型 ID、上下文窗口大小、是否支持流式输出、是否支持视觉、能力标签、每千 token 成本、状态（可用/不可用/已禁用）、性能统计（总调用次数、成功次数、失败次数、成功率、平均延迟、累计成本）。用户可以在该页面进行以下操作：添加新模型（点击添加模型按钮，填写模型配置信息，包括名称、API 地址、API Key、模型 ID、上下文窗口、是否支持视觉等，支持测试连接验证配置是否正确）、修改模型配置（点击模型条目进入编辑页面，修改各配置字段，API Key 修改后自动重新加密存储）、禁用启用模型（禁用后路由层不再选择该模型，但保留统计数据）、删除模型（从配置中移除模型提供商）、查看性能详情（点击模型查看详细的性能趋势图表）、手动切换当前模型（指定后续任务使用的模型，覆盖自动路由策略）。模型性能数据实时更新，每次模型调用后统计数据自动刷新。免密网络层的五家免费模型在模型管理页面以特殊标识显示，用户可以查看其状态和调用统计，但不能修改或删除。

技能管理页面展示已安装的所有技能列表，包括技能名称、描述、版本、作者、标签、触发条件、使用次数、安装来源（内置/市场/自定义）。用户可以查看技能详情、启用禁用技能、卸载技能、从技能市场搜索和安装新技能。技能市场兼容 `agentskills.io` 开放标准，涵盖前端开发、后端开发、DevOps、数据科学、安全审计、文档生成、内容创作等多个领域。内置技能包括内容摘要、中英互译、代码审查、小红书文案生成等，用户可以直接使用。自定义技能可以放置到技能目录中，软件启动时自动加载。

6.4 记忆与存储管理

记忆中心页面展示持久记忆系统的全部内容，包括 `MEMORY.md`（项目长期记忆）、`USER.md`（用户画像）、短期记忆状态、历史对话摘要列表。用户可以查看、编辑、删除记忆条目，手动添加重要记忆，触发记忆压缩和摘要生成。Honcho 记忆页面（v8.0 新增）展示当前用户

的画像信息（姓名、偏好、特质）、事实记忆列表（内容、重要度、创建时间、最后使用时间）、会话摘要历史、记忆检索测试工具（输入关键词测试检索结果）。用户可以手动添加事实记忆（指定内容和重要度）、编辑已有记忆、删除记忆、调整重要度评分、导出全部记忆数据。

存储统计页面（v8.0 新增）展示 SQLite 数据库的详细统计信息，包括数据库版本、数据库文件大小、WAL 日志大小、各数据表的记录数和占用空间、最近迁移记录、数据库健康状态。用户可以查看数据库表结构、执行数据备份（下载 feihong.db 文件）、触发数据清理（清理过期的任务记录和历史数据）、查看数据库操作日志。知识库页面管理用户上传的知识库文档，支持上传 Markdown、PDF、TXT 等格式的文档，自动解析和索引，代理在执行任务时可以检索知识库内容作为上下文参考。插件管理页面管理第三方插件，支持安装、启用、禁用、卸载插件，插件可以扩展代理的工具能力和界面功能。

6.5 API 接口

飞虹 Code 提供完整的 RESTful API 接口，支持第三方系统集成和自动化 workflow。核心 API 包括：健康检查 GET /api/health（公开，返回版本号、存储状态、Honcho 状态、时间）；对话接口 POST /api/chat（需认证，支持流式输出，参数：message、modelId、stream、context）；任务接口 GET/POST/DELETE /api/tasks（任务列表、创建、详情、删除）；模型接口 GET/POST/PUT/DELETE /api/models（模型配置管理，API Key 自动加密）；技能接口 GET/POST/DELETE /api/skills（技能管理）；记忆接口 GET/POST /api/memory（记忆管理）；存储统计 GET /api/storage/stats（v8.0 新增）；Honcho 接口 GET /api/honcho/context、POST /api/honcho/remember、GET /api/honcho/recall（v8.0 新增）；桥接回调接口 /api/wechat/callback、/api/feishu/callback、/api/yuanbao/callback（v8.0 新增）；Webhook 接口 POST /api/webhook/trigger（入站 Webhook 触发任务）。所有需认证的接口通过 Authorization: Bearer <FH_WEB_TOKEN> 请求头进行身份验证。API 文档通过 /api/docs 路径在线访问，采用 OpenAPI 风格，包含每个接口的请求参数、响应格式、错误码、示例代码。

第七章 高级功能

7.1 并行编排与团队协作

并行编排在需要同时处理多个独立子任务的场景下使用。并行编排器将用户的大目标拆解为多个相互独立的子任务，为每个子任务创建独立的 Git worktree 隔离工作区，并行启动多个子代理同时执行，最后汇总各工作区的执行结果。这种模式能够显著提升复杂任务的执行效率，特别适合涉及多个独立模块开发、多文件批量修改、多方案对比实验、多语言版本生成等场景。并行编排的执行流程分为五个步骤：目标拆解、工作区创建、并行执行（受最大并发数限制，默认 3）、结果汇总、工作区清理。

团队协作模式模拟人类开发团队的协作方式，创建多个具有不同角色和职责的代理成员（前端工程师、后端工程师、测试工程师、架构师等），共享一个任务清单，各成员自主认领任务并执行，通过消息总线进行通信协作，最终汇总产出团队报告。这种模式适用于需要多种专业能力协作完成的复杂任务，例如前端开发加后端开发加测试的全栈项目，或架构设计加代码实现加代码评审的质量保障流程。团队协作的核心组件包括任务看板（共享任务清单，支持原子认领）、消息总线（代理间通信机制，支持点对点和广播消息）、协调器（管理团队完整生命周期）。

7.2 MCP 集成与 GitHub MCP

MCP（Model Context Protocol，模型上下文协议）是飞虹 Code 支持的开放协议，允许代理连接外部 MCP 服务器，获取额外的工具和上下文资源。通过 MCP 集成，飞虹 Code 可以连接各种第三方服务和数据源，极大扩展代理的能力边界，例如连接数据库执行 SQL 查询、连接项目管理系统读取和更新任务、连接文档系统检索知识库、连接代码托管平台管理 Issue 和 PR、连接监控系统查询指标等。MCP 客户端实现了标准的 MCP 协议通信，支持多种传输方式，包括 stdio（标准输入输出，适用于本地 MCP 服务器进程）、SSE（Server-Sent Events，适用于远程 HTTP MCP 服务器）、HTTP（适用于 REST 风格的 MCP 服务器）。

MCP 客户端支持的核心操作包括工具列表查询、工具调用、资源读取、资源列表查询、提示模板获取。MCP 客户端自动处理协议握手、消息序列化、错误处理、连接管理、自动重连等底层细节，代理可以像调用内置工具一样透明地调用 MCP 工具。GitHub MCP Server（v8.0 新增）通过 `src/integrations/github-mcp.ts` 实现，专门用于连接 GitHub 平台的 MCP 服务器，代理可以调用 GitHub 相关工具（Issue 创建/查询/更新、PR 创建/审查/合并、代码搜索、仓库管理、提交查询、分支管理等），实现与 GitHub 平台的深度集成，例如自动创建 Issue、自动提交 PR、自动审查代码变更等。MCP 配置通过环境变量或配置文件进行，用户可以配置多个 MCP 服务器，代理启动时

自动连接所有配置的 MCP 服务器。MCP 集成的安全机制包括传输安全、认证安全、工具白名单、输入验证、输出过滤。

7.3 Webhook 集成

Webhook 功能允许飞虹 Code 接收外部系统的 HTTP 请求触发自动化任务（进站 Webhook），也可以在任务完成时向外部系统推送通知（出站 Webhook）。通过 Webhook 集成，飞虹 Code 可以与 CI/CD 流水线、项目管理系统、即时通讯工具、监报告警系统等无缝集成，实现自动化工作流。进站 Webhook：飞虹 Code 提供标准的 Webhook 接收端点 `POST /api/webhook/trigger`，外部系统可以通过 HTTP POST 请求触发自动化任务。进站 Webhook 支持多种认证方式确保请求来源可信：HMAC 签名校验、Bearer Token 认证、IP 白名单。进站 Webhook 的请求体包含任务触发信息，包括任务目标、执行模式、模型策略、回调 URL、自定义元数据等。服务端验证请求可信后创建任务并加入执行队列，立即返回任务 ID 和状态，任务在后台异步执行。

出站 Webhook：当任务状态变更或任务执行过程中产生重要事件时，飞虹 Code 可以向配置的回调 URL 推送 Webhook 通知。出站 Webhook 支持渠道白名单机制（`FH_CHANNEL_ALLOW` 环境变量），未在白名单中的 URL 不允许推送，防止数据外泄。出站 Webhook 的推送内容包含事件类型、任务 ID、任务状态、执行结果、错误信息、时间戳、HMAC 签名等。出站 Webhook 支持重试机制，当推送失败时自动重试，采用指数退避策略，最多重试 3 次。Webhook 集成的典型应用场景包括 CI/CD 集成（代码提交后自动触发代码评审和测试任务，结果回写到 PR 评论）、项目管理集成、即时通讯集成（在企业微信/钉钉/飞书群中通过机器人命令触发开发任务）、监报告警集成（监控系统检测到服务异常后自动触发故障排查任务）、自动化工作流。

7.4 企业级功能

飞虹 Code 提供完整的企业级功能，满足企业安全合规和团队协作需求。多租户隔离：支持多租户模式，每个租户的数据（任务、记忆、模型配置、知识库等）完全隔离，通过 `FH_TENANT_ID` 指定当前操作所属租户。RBAC 权限控制：基于角色的访问控制，支持管理员、开发者、审计员、只读用户等角色，每个角色有不同的权限范围，通过 `FH_RBAC_ENABLE` 启用。操作审计日志：记录系统中的所有操作事件，满足企业安全审计和合规要求，每条审计记录包含时间戳、操作用户、操作类型、操作详情、IP 地址、用户代理，支持多维度筛选、导出、长期归档、防篡改（追加写入和哈希校验），帮助企业满足等保 2.0、ISO 27001、SOC 2 等安全合规标准。资源配额管理：支持按租户或用户配置资源配额，包括任务并发数、模型调用次数、存储空间、执行时长等，防止资源滥用。企业级配置通过 `FH_ENTERPRISE=true` 启用，相关环境变量包括 `FH_TENANT_ID`、`FH_RBAC_ENABLE`、`FH_AUDIT_ENABLE`、`FH_QUOTA_ENABLE` 等。

第八章 配置参考

8.1 环境变量详解

飞虹 Code 通过环境变量进行运行时配置，所有环境变量均以 FH_ 前缀开头，便于识别和管理。环境变量可以通过多种方式设置：系统环境变量、.env 文件、命令行参数。配置加载优先级为命令行参数大于 .env 文件大于系统环境变量大于默认值。

环境变量	默认值	说明
FH_HOME	~/feihong-code	数据根目录（任务/记忆/知识库/插件/数据库）
FH_SECRET	自动生成	主密钥（用于加密敏感数据，>=16 字符）
FH_PROVIDERS	空	模型提供商 JSON 数组
FH_MODEL_NAME	空	默认模型名称
FH_MODEL_STRATEGY	cost	模型选择策略（cost/capability/latency）
FH_MAX_RETRIES	3	模型调用最大重试次数
FH_MAX_CYCLES	50	代理最大执行轮次
FH_OFFLINE	false	强制离线模式
FH_WEB_PORT	8080	Web 控制台端口
FH_WEB_HOST	0.0.0.0	Web 控制台监听地址
FH_WEB_TOKEN	自动生成	Web 控制台访问令牌
FH_TASK_CONCURRENCY	2	任务队列并发数
FH_LOG_LEVEL	info	日志级别（debug/info/warn/error）
FH_DB_PATH	\$FH_HOME/feihong.db	SQLite 数据库路径（v8.0）
FH_HONCHO_USER	default	Honcho 当前用户 ID（v8.0）

FH_HONCHO_URL	空	外部 Honcho 服务 URL (v8.0, 预留)
FH_SANDBOX_IMAGE	node:22-alpine	Docker 沙盒镜像 (v8.0)
FH_SANDBOX_NETWORK	false	沙盒是否启用网络 (v8.0)
FH_SANDBOX_READONLY	true	沙盒工作区只读挂载 (v8.0)
FH_SANDBOX_TIMEOUT	60000	沙盒执行超时毫秒 (v8.0)
FH_ENTERPRISE	false	启用企业级功能
FH_TENANT_ID	default	当前租户 ID
FH_CHANNEL_ALLOW	空	出站渠道白名单

8.2 多平台桥接配置

微信桥接配置：FH_WECHAT_MODE (wecom/mp)、FH_WECHAT_CORP_ID、FH_WECHAT_AGENT_ID、FH_WECHAT_SECRET、FH_WECHAT_TOKEN、FH_WECHAT_AES_KEY、FH_WECHAT_APP_ID、FH_WECHAT_APP_SECRET。飞书桥接配置：FH_FEISHU_ENABLED、FH_FEISHU_APP_ID、FH_FEISHU_APP_SECRET、FH_FEISHU_VERIFICATION_TOKEN、FH_FEISHU_ENCRYPT_KEY。元宝桥接配置：FH_YUANBAO_ENABLED、FH_YUANBAO_APP_ID、FH_YUANBAO_APP_SECRET。GitHub MCP 配置：FH_GITHUB_TOKEN (GitHub 个人访问令牌，用于 GitHub MCP Server 认证)、FH_GITHUB_MCP_URL (GitHub MCP 服务器地址，可选，默认使用官方服务器)。所有桥接功能配置后，对应回调地址需要在各平台后台配置，回调地址格式为 <https://域名/api/平台名/callback>。

第九章 常见问题与故障排查

9.1 安装与启动问题

问题一：npm install -g feihong-code 报权限错误（EACCES）。解决方案：macOS/Linux 使用 `sudo npm install -g feihong-code`；Windows 以管理员身份运行终端；推荐修改 npm 全局安装目录到用户主目录下（`npm config set prefix ~/.npm-global`，将 `~/.npm-global/bin` 添加到 PATH）。

问题二：执行 fhcode 命令提示 `command not found`。解决方案：确认安装成功（`npm list -g feihong-code`）；将 npm 全局安装目录的 bin 子目录添加到 PATH 环境变量；重启终端使环境变量生效。

问题三：启动时报错 `node:sqlite 模块未找到`。原因分析：Node.js 版本低于 22，node:sqlite 是 Node.js 22+ 内置模块。解决方案：升级 Node.js 到 22.0.0 及以上版本（推荐 Node.js 22 LTS），执行 `node --version` 确认版本，重新编译（`npm run build`）。

问题四：启动后提示未配置任何模型进入免密模式。说明：这是正常行为，v8.0 版本未配置 FH_PROVIDERS 时自动进入免密网络层模式，使用内置五家免费模型环形轮转，无需任何 API Key 即可使用。如需使用自定义模型，按照模型配置章节设置 FH_PROVIDERS 环境变量。

9.2 模型调用问题

问题一：模型调用返回 401 Unauthorized。解决方案：检查 API Key 是否正确（注意复制时不要包含多余空格或换行）；确认 API Key 未过期且账户有足够额度；确认 apiBase 地址与 API Key 匹配；重新生成 API Key 并更新配置。

问题二：模型调用返回 429 Too Many Requests。解决方案：降低 FH_TASK_CONCURRENCY 并发数；配置多个模型提供商，路由层自动轮换分散请求压力；等待一段时间后重试（路由层内置指数退避）；升级 API 套餐或使用本地 Ollama 模型。免密网络层模式下会自动切换到下一个免费模型。

问题三：对话只有一轮就停止生成。原因分析：上下文超限、模型 API 限流、网络超时、APP 端配置问题。解决方案：使用 /clear 清空上下文开始新对话；减少注入的上下文文件大小；选择支持更长上下文的模型；检查网络连接；确认 API Key 有效且有额度；在 APP 端检查自定义模型配置是否正确。

问题四：图片上传后无法识别。解决方案：需要配置支持视觉的模型（如 DeepSeek-OCR、GPT-4o 等），在模型配置中将 supportsVision 设置为 true；确认图片大小在限制范围内（默认

10MB)；确认模型 API 支持图片输入格式。

9.3 Docker 沙盒问题

问题一：Docker 沙盒功能不可用。解决方案：确认本机已安装 Docker Desktop 或 Docker Engine (20.10+)；执行 `docker info` 确认 daemon 在运行（Windows 用户需启动 Docker Desktop）；确认当前用户有 Docker 访问权限（Linux 用户可能需要将用户加入 docker 组）。

问题二：Docker Desktop 启动报错 `invalid character "i" looking for beginning of value`。原因分析：daemon.json 文件包含 UTF-8 BOM（字节顺序标记），Docker JSON 解析器无法识别。解决方案：用 VS Code 打开 daemon.json，右下角编码选择"UTF-8 without BOM"保存；或删除该文件让 Docker 自动重建；或用 .NET [System.IO.File]::WriteAllText 配合 UTF8Encoding(\$false) 无 BOM 重写。daemon.json 路径：Windows 为 `C:\Users\用户名\.docker\daemon.json`。

问题三：拉取镜像超时或失败（failed to resolve reference）。原因分析：Docker Hub 在国内无法直接访问。解决方案：配置国内镜像加速器（编辑 daemon.json 的 registry-mirrors，添加 `docker.1ms.run`、`docker.m.daocloud.io`、`hub-mirror.c.163.com`、`mirror.baidubce.com` 等），重启 Docker Desktop 后重新拉取；或手动通过加速器拉取后重新打标签（`docker pull docker.1ms.run/library/node:22-alpine && docker tag docker.1ms.run/library/node:22-alpine node:22-alpine`）。

9.4 数据存储与记忆问题

问题一：SQLite 数据库文件在哪里。答案：默认在 `~/feihong-code/feihong.db`（Windows 为 `C:\Users\用户名\feihong-code\feihong.db`），可通过 `FH_DB_PATH` 环境变量自定义路径。

问题二：模型 API Key 是明文存储吗。答案：不是。SQLite 存储中模型 API Key 自动 AES-256-GCM 加密落盘，主密钥来自 `FH_SECRET` 环境变量或 `$FH_HOME/.secret` 文件。modelGet 接口返回解密后的明文供调用方使用，数据库中存储的是加密后的密文。

问题三：如何重置所有数据。警告：此操作不可恢复。解决方案：停止服务后删除 `~/feihong-code/` 目录（包含任务、记忆、知识库、SQLite 数据库、主密钥等全部数据），重启服务后自动重新初始化。

问题四：Honcho 记忆不生效。解决方案：确认 `/api/health` 返回 `honcho.ok` 为 true；确认 `FH_HONCHO_USER` 设置正确；通过 `POST /api/honcho/remember` 手动添加一条测试记忆，然后通过 `GET /api/honcho/recall` 验证检索；查看日志确认记忆注入提示词是否生成。

9.5 移动端 APP 问题

问题一：APP 安装后闪退。解决方案：检查 Android 版本（要求 Android 8.0+）；卸载后重新安装最新版 APK；检查手机存储空间是否充足。

问题二：游戏画面超出屏幕或点击无反应。解决方案：更新到最新版 APK（v8.0 已优化游戏适配，所有游戏针对手机屏幕尺寸和触摸操作进行了专门适配）；在游戏内使用适配屏幕按钮；尝试横竖屏切换。

问题三：APP 对话没有回复，提示已停止生成。解决方案：检查自定义模型配置是否正确（API 地址、API Key、模型 ID）；确认 API Key 有效且有额度；切换到免密网络层模式（不配置自定义模型时自动启用）；检查网络连接；更新到最新版 APK。

问题四：文生图/文生视频按钮和退出按钮叠加在一起。解决方案：更新到最新版 APK（v8.0 已优化创作界面布局，退出按钮和功能按钮有明显的间距分隔）；清除 APP 缓存后重新打开。

第十章 详细教程与实战案例

10.1 案例一：从零创建一个 Web 应用

场景描述：用户希望创建一个基于 Express.js 的待办事项 (Todo) Web 应用，包含增删改查功能和简单的前端界面。操作步骤：第一步，启动飞虹 Code (fhcode serve 启动 Web 控制台，或 fhcode 启动 CLI 交互模式)。第二步，输入需求"创建一个 Express.js 待办事项应用，包含 RESTful API (GET/POST/PUT/DELETE /api/todos)，数据存储在 SQLite 数据库，前端使用原生 HTML/CSS/JavaScript，支持添加、编辑、删除、标记完成待办事项"。第三步，代理自动进行任务规划，拆解为以下子任务：初始化项目结构 (package.json、目录结构)、安装依赖 (express、better-sqlite3 或使用 node:sqlite)、创建数据库模块 (数据库连接、表创建、CRUD 操作封装)、创建 API 路由 (四个端点的实现)、创建前端界面 (HTML 结构、CSS 样式、JavaScript 交互逻辑)、创建启动入口 (server.js)、编写 README 文档。第四步，代理依次执行每个子任务，调用文件写入工具创建各文件，调用 Shell 工具执行 npm install，调用构建验证工具确认依赖安装成功。第五步，任务完成后，代理输出总结报告，包含创建的文件清单、启动命令 (npm start)、访问地址 (http://localhost:3000)。用户可以直接运行应用验证功能。整个过程无需人工编写任何代码，代理自主完成从需求理解到代码实现的全流程。

10.2 案例二：代码评审与自动修复

场景描述：用户有一个已有的 Node.js 项目，希望对代码进行全面评审，发现问题并自动修复。操作步骤：第一步，在项目根目录启动飞虹 Code。第二步，输入需求"对当前项目进行全面代码评审，重点关注安全漏洞、性能问题、代码规范、错误处理，输出评审报告并自动修复发现的问题"。第三步，代理调用代码评审工具，扫描项目中的所有源代码文件，基于预定义规则集检测各类问题，按严重程度 (critical/high/medium/low) 分类，生成结构化评审报告。第四步，代理根据评审报告，自动修复发现的问题：安全漏洞 (如 SQL 注入风险改为参数化查询、XSS 风险添加输入转义、硬编码密钥移到环境变量)、性能问题 (如同步阻塞操作改为异步、重复计算添加缓存、不必要的循环优化)、代码规范 (如命名规范、注释补充、代码格式)、错误处理 (如添加 try-catch、错误信息规范化、未处理的 Promise rejection 处理)。第五步，修复完成后，代理自动运行构建和测试验证修复没有引入新错误，输出修复前后的对比报告和修改文件清单。用户可以通过 git diff 查看所有变更，确认无误后提交。

10.3 案例三：使用 Docker 沙盒执行不可信代码

场景描述：用户需要执行一段来自不可信来源的 Python 代码，担心安全风险，希望在隔离环境中执行。操作步骤：第一步，确认 Docker 已安装并运行（docker info）。第二步，在飞虹 Code 中输入需求"执行这段 Python 代码，使用 Docker 沙盒隔离执行，禁止网络访问，限制内存 256MB：[代码内容]"。第三步，代理调用 Docker 沙盒工具，自动执行以下操作：创建临时工作目录，将代码写入工作目录；启动 Docker 容器（使用 python:3.11-alpine 镜像，--rm 一次性容器，--network none 禁用网络，-m 256m 内存限制，--read-only 只读根文件系统，工作区只读挂载）；在容器内执行 Python 代码；捕获标准输出和标准错误；超时（默认 60 秒）后自动 docker kill 强杀；清理容器和临时文件。第四步，代理返回代码执行结果（标准输出、标准错误、退出码、执行耗时）。整个执行过程在完全隔离的容器中进行，代码无法访问网络、无法修改宿主主机文件、资源使用受限，即使代码包含恶意操作（如文件删除、网络请求、挖矿程序）也不会影响宿主机安全。

10.4 案例四：利用持久记忆实现个性化开发

场景描述：用户长期使用飞虹 Code 开发多个项目，希望代理能够记住用户的编码习惯、技术栈偏好、项目上下文，无需每次重新交代背景。操作步骤：第一步，首次使用时，用户在对话中自然表达偏好（如"我习惯用 TypeScript 写后端，用 React 写前端，代码风格偏向函数式，注释要详细"）。第二步，Honcho 语义记忆系统自动从用户消息中提取偏好类事实，沉淀到用户画像（users 表）和事实记忆（user_memory 表），附带重要度评分（如"偏好 TypeScript"重要度 0.9，"注释要详细"重要度 0.7）。第三步，后续对话中，代理自动检索用户画像和相关事实记忆，生成个性化上下文提示词，注入到系统提示中。例如用户说"创建一个新的 API 服务"，代理会自动选择 TypeScript、应用用户的代码风格、添加详细注释，无需用户再次说明。第四步，项目上下文自动记录到 MEMORY.md，包括项目技术栈、目录结构、关键决策、历史任务摘要。跨会话时代理自动加载项目上下文，理解当前项目状态。第五步，用户可以通过 /memory 命令查看记忆状态，通过 Web 控制台记忆中心管理记忆内容，手动调整重要度或删除不准确的记忆。随着使用时间增长，代理对用户的理解不断加深，开发效率和个性化程度持续提升。

附录 A 命令速查表

命令	说明
fhcode --version	显示版本号
fhcode --help	显示帮助信息
fhcode	启动交互式 REPL 对话
fhcode "需求"	单次执行任务（非交互式）
fhcode serve	启动 Web 控制台（默认端口 8080）
fhcode serve --port 9090	指定端口启动 Web 控制台
fhcode review [路径]	代码评审
fhcode run "目标"	执行单次任务
/plan	REPL 中切换规划模式
/grill	REPL 中深度拷问式审查
/memory	REPL 中查看记忆状态
/skills	REPL 中列出已安装技能
/models	REPL 中列出已配置模型
/status	REPL 中查看 Agent 状态
/clear	REPL 中清空对话上下文
/exit	退出 REPL
npm run build	编译 TypeScript 到 dist/
node tests/smoke-v8.js	运行 v8.0 冒烟测试（25 项）
node tests/integration/api.test.js	运行 API 集成测试（34 项）

附录 B 配置速查表

配置项	位置	说明
模型提供商	FH_PROVIDERS 环境变量 / Web 控制台模型管理	配置一个或多个模型提供商
Web 控制台端口	FH_WEB_PORT / --port	默认 8080
访问令牌	FH_WEB_TOKEN / --token	生产环境建议配置
数据库路径	FH_DB_PATH	默认 ~/.feihong-code/feihong.db
主密钥	FH_SECRET / ~/.feihong-code/.secret	用于加密敏感数据
Honcho 用户	FH_HONCHO_USER	默认 default
Docker 沙盒镜像	FH_SANDBOX_IMAGE	默认 node:22-alpine
沙盒网络	FH_SANDBOX_NETWORK	默认 false (禁用)
沙盒超时	FH_SANDBOX_TIMEOUT	默认 60000 毫秒
任务并发数	FH_TASK_CONCURRENCY	默认 2
日志级别	FH_LOG_LEVEL	debug/info/warn/error
企业级功能	FH_ENTERPRISE	true 启用多租户/RBAC/审计
微信桥接	FH_WECHAT_* 系列	wecom/mp 双模式
飞书桥接	FH_FEISHU_* 系列	飞书机器人接入
出站白名单	FH_CHANNEL_ALLOW	逗号分隔的 URL 模式

附录 C 术语表

术语	说明
Agent (代理)	由大语言模型驱动的自主执行实体，能够理解目标、规划任务、调用工具、迭代执行
SWE Agent	软件工程代理，采用规划-执行-验证-修复闭环架构处理复杂开发任务
多模型路由	同时配置多个模型提供商，按策略自动选择最优模型，故障时自动轮换
免密网络层	内置五家免费模型环形轮转+故障转移，无需 API Key 即可使用
Ring Failover	环形故障转移，模型故障时自动切换到环形队列中的下一个可用模型
持久记忆	跨会话的长期记忆系统，包括项目记忆、用户画像、Honcho 语义记忆
Honcho	语义记忆系统，实现用户建模、事实记忆、跨会话检索、记忆注入
SQLite 存储	基于 node:sqlite 的统一数据存储层，十张表，AES 加密，版本化迁移
Docker 沙盒	在 Docker 容器中隔离执行代码，禁网/只读/资源限制/危险命令拦截
MCP	Model Context Protocol，模型上下文协议，连接外部工具和数据源的开放标准
Webhook	通过 HTTP 请求触发任务或推送通知的机制，实现系统间集成
技能 (Skill)	可复用的专业能力配置，兼容 agentskills.io 标准，可自动提炼和安装
自进化	代理从任务执行中自动积累经验，持续提升执行能力和效率
RBAC	基于角色的访问控制，企业级权限管理
工作树 (Worktree)	Git 的并行工作区机制，用于并行编排中隔离各子代理的工作环境
上下文压缩	对话过长时自动将历史摘要为紧凑形式，减少 token 占用
自我修复	构建或测试失败时自动分析错误原因并迭代修复，直至成功

附录 D 更新日志

V8.0.0 (2026年9月3日)

- 新增: SQLite 统一数据存储 (十张表, AES-256-GCM 加密模型 Key, 版本化迁移, 零外部依赖)
- 新增: Docker 沙盒执行 (容器隔离/禁网/只读挂载/资源限制/危险命令拦截/超时强杀)
- 新增: Honcho 语义记忆 (用户建模/事实记忆/跨会话检索/记忆注入/自动沉淀)
- 新增: 免密网络层 (五家免费模型环形轮转+Ring Failover, 开箱即用零配置)
- 新增: 多平台消息桥接 (微信/飞书/元宝/GitHub MCP Server)
- 新增: Android APP (对话/游戏中心/文生图/文生视频/图生视频/自定义模型/视觉模型)
- 新增: 自动化调度 (自然语言定时任务, 替代 cron)
- 新增: SessionStart/PostToolUse/PreCompact 钩子支持
- 新增: pua-ext/frontend-dev-guide/backend-dev-guide 技能接入
- 新增: VoltAgent/awesome-agent-skills 技能市场接入
- 优化: SWE-bench 源文件定位率提升、no_patch 分析、多模型路由、贝叶斯自动调参
- 优化: Web 控制台界面布局、创作界面按钮间距、游戏适配
- 修复: 移动端对话多轮上下文、游戏屏幕适配、API Key 硬编码安全问题
- 文档: 详细使用说明书 v8.0.0、技术说明书 v2.0、升级说明书、项目分析与测试说明书 v2.0
- 社区: GOVERNANCE.md、COMMUNITY.md、Issue/PR 模板、贡献指南更新

V7.9.0 (2026年9月3日)

- 新增: Hermes Agent 框架 (持久记忆/自演化技能/多平台网关/沙盒执行/自动化调度)
- 新增: LLM 辅助技能提炼
- 新增: 集成测试套件 (33 项)
- 新增: API 文档 (OpenAPI 风格)
- 新增: 日志轮转 (log-rotator)

V7.6.0 (2026年8月28日)

- 新增: SWE Agent 全自动软件工程代理
- 新增: 多 Agent 协作 (Solo/Multi/Event-Driven/Custom)

- 新增：知识库管理
- 优化：企业级 RBAC 审计

V7.5.0 (2026年8月27日)

- 新增：企业级 RBAC 权限控制
- 新增：操作审计日志
- 新增：多租户隔离
- 新增：资源配额管理

V7.2.0 (2026年8月26日)

- 初始版本：基础架构、多模型路由、Web 控制台、CLI 交互
 - 新增：代码评审引擎
 - 新增：技能系统
 - 新增：自进化与经验学习
 - 新增：并行编排与团队协作
 - 新增：MCP 集成与 Webhook 集成
-